Índice

1. Scripts de Carga y Limpieza (Data Provisioning)
    1.1 Verificación del entorno de ejecución e instalación de dependencias con npm ci
    1.2 Configuración de conexión con MongoDB Atlas
    1.3 Verificación de conexiones de los microservicios con MongoDB Atlas
    1.4 Verificación del estado inicial de la base de datos
    1.5 Figura 1.5. Ejecución del script de carga de datos
        1.5.1 Figura 1.5.1. Verificación de los datos insertados en MongoDB Atlas
    1.6 Figura 1.6. Verificación de persistencia mediante consultas a MongoDB
    1.7 Figura 1.7. Ejecución del script de limpieza de datos
    1.8 Figura 1.8. Verificación de la limpieza mediante consultas a MongoDB
2. Capturas o Logs de Ejecución de Queries
    2.1 Figura 2.1 Consulta de usuarios registrados mediante la API REST de FIXIT
    2.2 Figura 2.2. Verificación de autenticación y recuperación de usuario mediante la PWA
        2.2.1 Figura 2.2.1. Verificación de recuperación del perfil de usuario mediante la API REST
    2.3 Figura 2.3. Consulta del estado inicial de sesiones mediante la API REST
        2.3.1 Figura 2.3.1. Comprobación del estado inicial en MongoDB Atlas
        2.3.2 Figura 2.3.2. Registro de una sesión de juego mediante la API REST de FIXIT
        2.3.3 Figura 2.3.3. Comprobación de la persistencia de una sesión de juego en MongoDB Atlas
        2.3.4 Figura 2.3.4. Verificación del resumen actualizado de sesiones mediante la API REST
    2.4 Figura 2.4. Verificación de estadísticas actualizadas mediante User Service
        2.4.1 Figura 2.4.1. Comprobación de estadísticas persistidas en MongoDB Atlas
        2.4.2 Figura 2.4.2. Consulta del progreso del usuario mediante Progress Service
        2.4.3 Figura 2.4.3. Comprobación del progreso persistido en MongoDB Atlas
    2.5 Figura 2.5. Verificación de la recuperación de estadísticas persistidas desde la PWA FIXIT
        2.5.2 Figura 2.5.2. Verificación de la recuperación del resumen de sesiones desde la PWA FIXIT
        2.5.3 Figura 2.5.3. Verificación de la recuperación del progreso del usuario desde la PWA FIXIT

Scripts de Carga y Limpieza (Data Provisioning):

1.1 Verificación del entorno de ejecución e instalacion de dependencias con npm ci

```
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> node --version ; npm --version
v24.22.0
11.19.1
```

```
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> npm ci

added 20 packages, and audited 21 packages in 2s

2 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> npm ls mongoose dotenv --depth=0
fixit-backend-tools@1.0.0 C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend
+-- dotenv@17.4.2
`-- mongoose@9.11.2

PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> Copy-Item .env.example .env
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> Test-Path .\.env
True
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend>
```

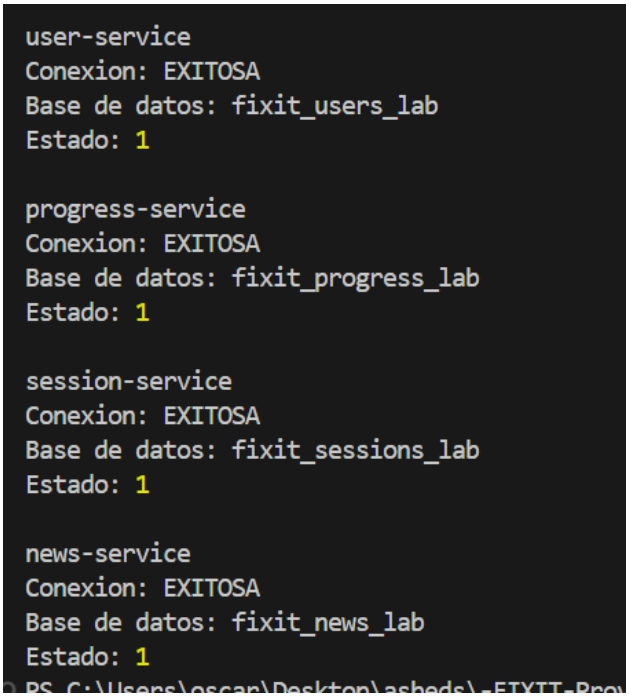
Se verificó el entorno de ejecución del backend de FIXIT mediante PowerShell, comprobando las versiones instaladas de Node.js y npm, la disponibilidad de los scripts de carga y limpieza, así como las dependencias necesarias para establecer la conexión con MongoDB Atlas.

1.2 Configuración de conexión con MongoDB Atlas

```
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> Test-Path .env
>> Test-Path user-service/.env
>> Test-Path progress-service/.env
>> Test-Path session-service/.env
>> Test-Path news-service/.env
True
True
True
True
True
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> node --input-type=module -e "import 'dotenv/config'; const uri=proc
ess.env.MONGO_URI; if (!uri) { console.log('MONGO_URI: no configurada'); process.exit(1); } try { const u=new URL(uri); console.log('Pro
tocolo:',u.protocol); console.log('Base de datos:',decodeURIComponent(u.pathname.slice(1)) || '(sin especificar)'); console.log('URI con
figurada: Sí'); } catch { console.log('URI inválida'); process.exitCode=1; }"
Protocolo: mongodb:
Base de datos: fixit
URI configurada: Sí
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend>
```

Se verificó la configuración de conexión del backend de FIXIT mediante PowerShell, comprobando la existencia de los archivos `.env` correspondientes a los cuatro microservicios y al aprovisionador de datos. Asimismo, se identificó la base de datos `fixit` como destino configurado para las operaciones de persistencia y administración de información.

1.3 Verificación de conexiones de los microservicios con MongoDB Atlas

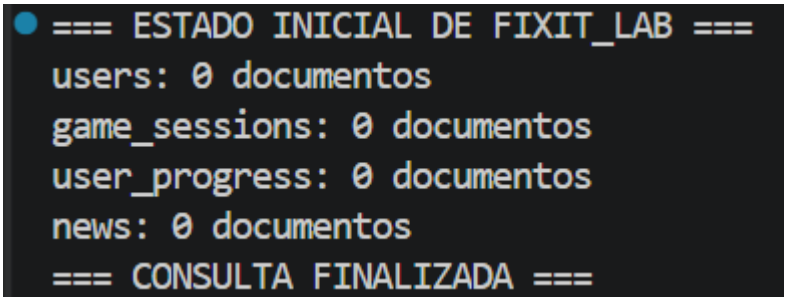


Se verificó exitosamente la conexión de los cuatro microservicios del backend de FIXIT con MongoDB Atlas mediante Node.js y Mongoose. Durante la ejecución se comprobó el acceso independiente a las bases de datos `fixit_users_lab`, `fixit_progress_lab`, `fixit_sessions_lab` y `fixit_news_lab`, obteniendo en todos los casos el estado de conexión 1, lo que confirma que las conexiones se establecieron correctamente y que los servicios cuentan con acceso a sus respectivas bases de datos. Se colocó la terminacion "_lab" con el fin de crear instancias exclusivas para el laboratorio de esta asignación.

NOTA: comando utilizado: -FIXIT-Proyecto-Integrador\backend> @'
>> import { readFileSync } from "node:fs";
>> import { parse } from "dotenv";
>> import mongoose from "mongoose";
>>
>> const services = [
>> "user-service",
>> "progress-service",
>> "session-service",
>> "news-service"
>>];
>>
>> for (const service of services) {
>> let connection;
>>
>> try {
>> const env = parse(readFileSync(`${service}/.env`));
>> const uri = env.MONGO_URI;

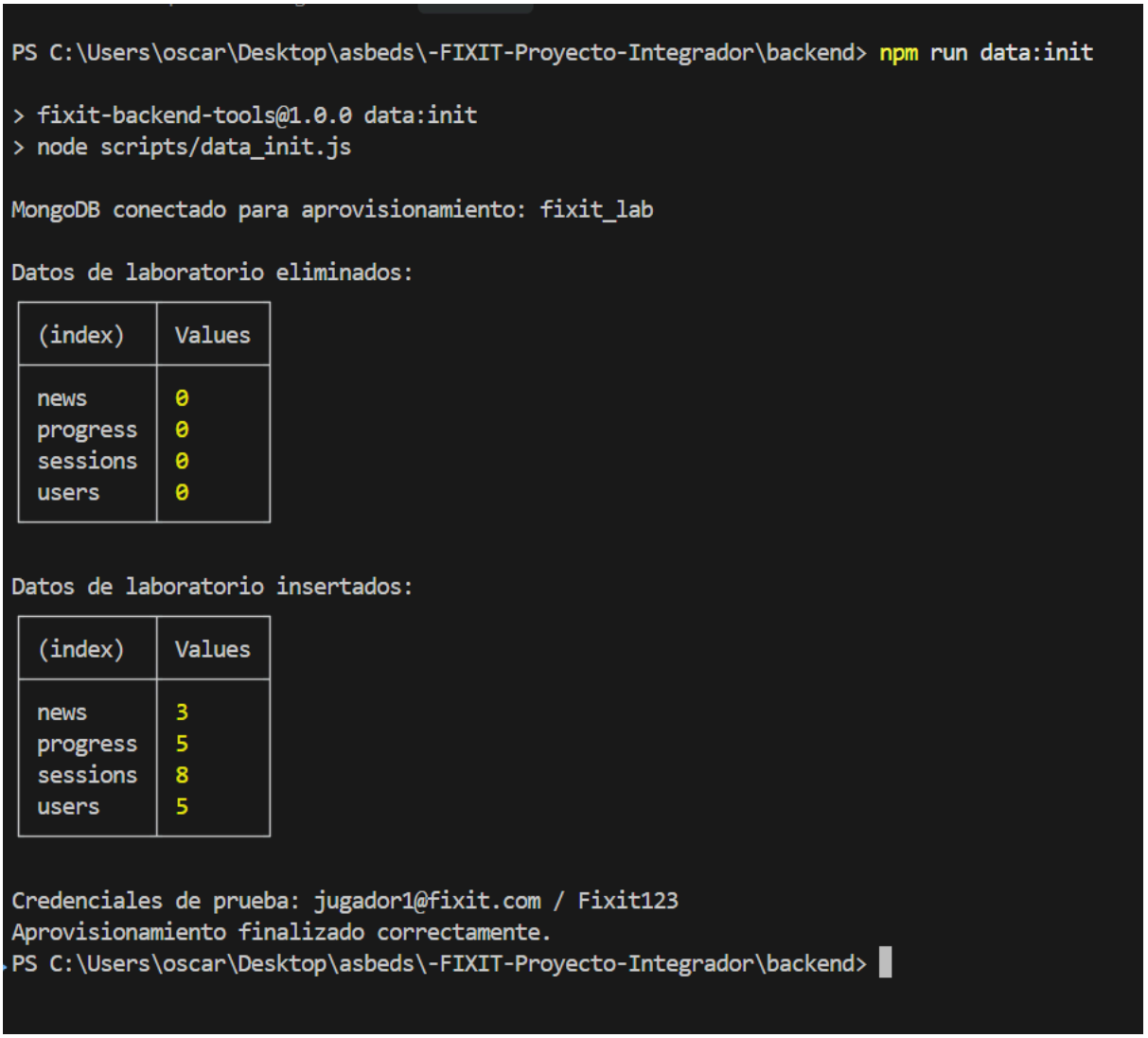
```
>>
>>   if (!uri) {
>>     throw new Error("MONGO_URI no configurada");
>>   }
>>
>>   connection = await mongoose
>>     .createConnection(uri, {
>>       serverSelectionTimeoutMS: 10000
>>     })
>>     .asPromise();
>>
>>   console.log(`\n${service}`);
>>   console.log("Conexion: EXITOSA");
>>   console.log("Base de datos:", connection.name);
>>   console.log("Estado:", connection.readyState);
>>
>> } catch (error) {
>>   console.log(`\n${service}`);
>>   console.log("Conexion: FALLIDA");
>>   console.log("Error:", error.message);
>> } finally {
>>   if (connection) {
>>     await connection.close();
>>   }
>> }
>> }
>> '@ | node --input-type=module
```

1.4 Verificación del estado inicial de la base de datos



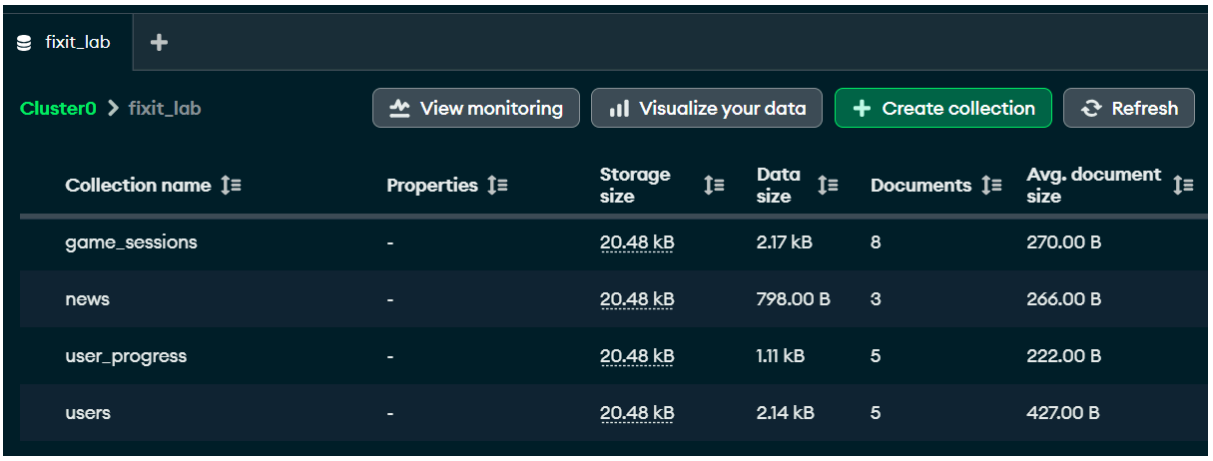
Mongoose. Se verificó que las colecciones `users`, `game_sessions`, `user_progress` y news contenían cero documentos antes de ejecutar los scripts de aprovisionamiento, estableciendo así un estado inicial de referencia para comprobar posteriormente la inserción y persistencia de los datos de prueba.

Siguiente: Figura 1.5. Ejecución del script de carga de datos



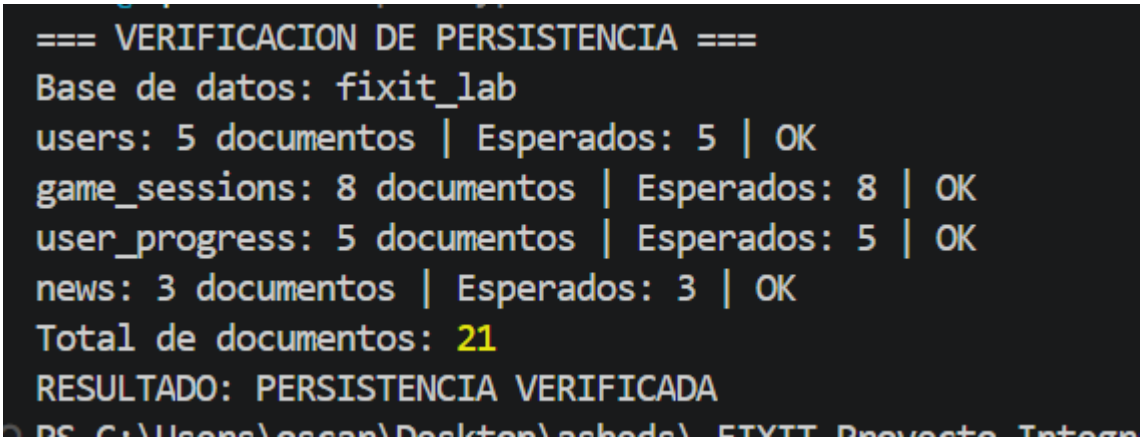
Se ejecutó exitosamente el script de aprovisionamiento `data_init.js` mediante el comando `npm run data:init`, estableciendo conexión con la base de datos `fixit_lab` en MongoDB Atlas. Durante el proceso se realizó una limpieza inicial de los registros reservados para el laboratorio y posteriormente se insertaron 21 documentos, distribuidos en 5 usuarios, 8 sesiones de juego, 5 registros de progreso y 3 noticias. La ejecución finalizó correctamente, demostrando el funcionamiento del mecanismo automatizado de carga de datos de prueba.

Figura 1.5.1. Verificación de los datos insertados en MongoDB Atlas



Se verificó mediante la interfaz de MongoDB Atlas que los datos insertados por el script `data_init.js` se almacenaron correctamente en la base de datos `fixit_lab`, alojada en el clúster `Cluster0`. Se confirmó la existencia de cuatro colecciones: `users` con 5 documentos, `game_sessions` con 8, `user_progress` con 5 y news con 3, sumando un total de 21 registros. Esta evidencia complementa la ejecución del script de carga y demuestra que las inserciones se reflejaron correctamente en la base de datos.

Figura 1.6. Verificación de persistencia mediante consultas a MongoDB



Se realizaron consultas de verificación a la base de datos `fixit_lab` mediante Node.js y Mongoose, comprobando la persistencia de los registros insertados previamente por el script `data_init.js`. Los resultados confirmaron la existencia de 5 documentos en `users`, 8 en `game_sessions`, 5 en `user_progress` y 3 en news, obteniendo un total de 21 documentos. Todas las consultas coincidieron con las cantidades esperadas, demostrando que la información permaneció almacenada correctamente en MongoDB Atlas después de finalizar el proceso de carga.

Comando utilizado:
\-FIXIT-Proyecto-Integrador\backend> @'
>> import "dotenv/config";
>> import mongoose from "mongoose";
>>

```
>> try {
>>   await mongoose.connect(process.env.MONGO_URI, {
>>     serverSelectionTimeoutMS: 10000
>>   });
>>
>>   if (mongoose.connection.name !== "fixit_lab") {
>>     throw new Error("La base configurada no es fixit_lab");
>>   }
>>
>>   console.log("=== VERIFICACION DE PERSISTENCIA ===");
>>   console.log("Base de datos:", mongoose.connection.name);
>>
>>   const expected = {
>>     users: 5,
>>     game_sessions: 8,
>>     user_progress: 5,
>>     news: 3
>>   };
>>
>>   let total = 0;
>>   let valid = true;
>>
>>   for (const [name, expectedCount] of Object.entries(expected)) {
>>     const count = await mongoose.connection.db
>>       .collection(name)
>>       .countDocuments();
>>
>>     total += count;
>>
>>     const ok = count === expectedCount;
>>     if (!ok) valid = false;
>>
>>     console.log(
>>       `${name}: ${count} documentos | Esperados: ${expectedCount} | ${ok ? "OK" :
"REVISAR"}`
>>     );
>>   }
>>
>>   console.log("Total de documentos:", total);
>>   console.log(
>>     valid
>>       ? "RESULTADO: PERSISTENCIA VERIFICADA"
>>       : "RESULTADO: EXISTEN DIFERENCIAS"
>>   );
>>
>> } catch (error) {
>>   console.error("Error:", error.message);
>>   process.exitCode = 1;
```

```
>> } finally {
>>   await mongoose.disconnect();
>> }
>> '@ | node --input-type=module
```

Figura 1.7. Ejecución del script de limpieza de datos

```
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador\backend> npm run data:clean

> fixit-backend-tools@1.0.0 data:clean
> node scripts/data_init.js --clean-only

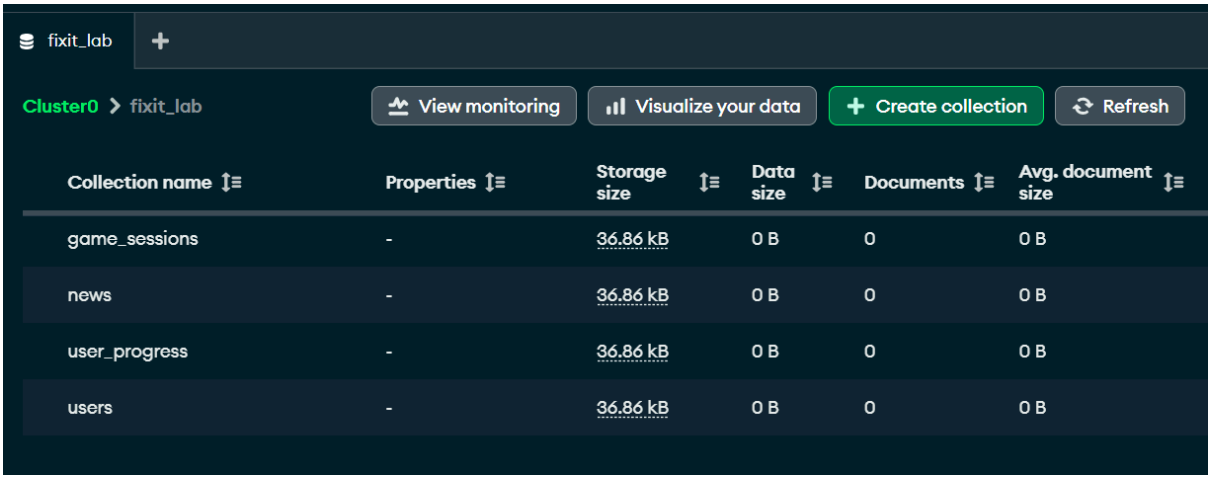
MongoDB conectado para aprovisionamiento: fixit_lab

Datos de laboratorio eliminados:
```

(index)	Values
news	3
progress	5
sessions	8
users	5

```
Limpieza finalizada. No se cargaron datos nuevos.
```

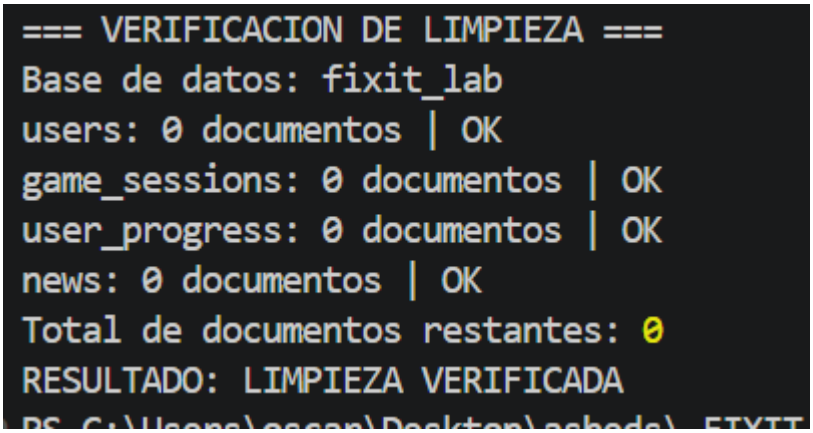
Se ejecutó exitosamente el script de limpieza mediante el comando `npm run data:clean`, estableciendo conexión con la base de datos `fixit_lab` en MongoDB Atlas. Durante el proceso se eliminaron 21 documentos de laboratorio, distribuidos en 5 usuarios, 8 sesiones de juego, 5 registros de progreso y 3 noticias. La ejecución finalizó correctamente sin insertar nuevos registros, demostrando el funcionamiento del mecanismo automatizado de limpieza y administración de datos de prueba.



Se verificó mediante la interfaz de MongoDB Atlas que el script de limpieza `data_init.js`, ejecutado mediante el comando `npm run data:clean`, eliminó correctamente los 21 documentos de prueba almacenados en la base de datos `fixit_lab`. Se comprobó que las colecciones `users`, `game_sessions`, `user_progress` y `news` permanecieron disponibles, pero con cero documentos registrados. Esta evidencia confirma la eliminación

de los datos de laboratorio sin afectar la estructura de las colecciones, demostrando el funcionamiento del proceso automatizado de limpieza.

Figura 1.8. Verificación de la limpieza mediante consultas a MongoDB



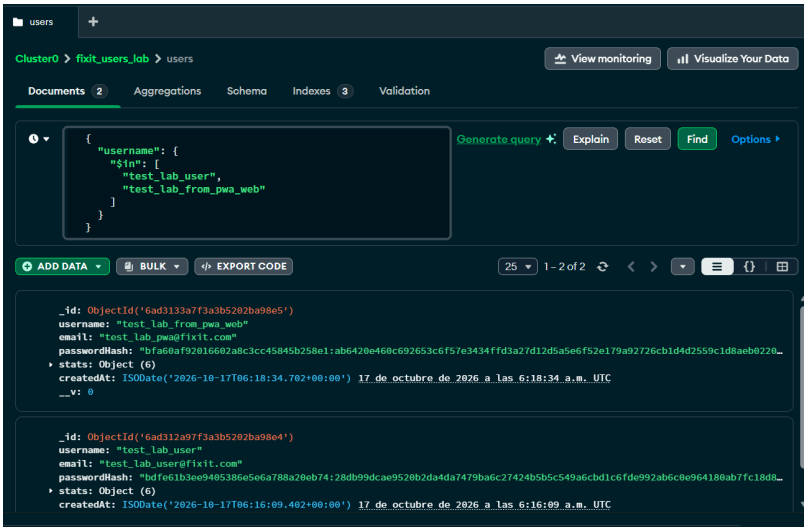
Se realizaron consultas de verificación a la base de datos `fixit_lab` mediante Node.js y Mongoose, con el propósito de comprobar los resultados del script automatizado de limpieza. Se confirmó que las colecciones `users`, `game_sessions`, `user_progress` y `news` contenían cero documentos, obteniendo un total de cero registros restantes. Todas las consultas devolvieron resultados satisfactorios, demostrando que los 21 documentos de prueba fueron eliminados correctamente y que la base de datos quedó libre de los registros utilizados durante el laboratorio.

Capturas o Logs de Ejecución de Queries

Figura 2.1 Consulta de usuarios registrados mediante la API REST de FIXIT



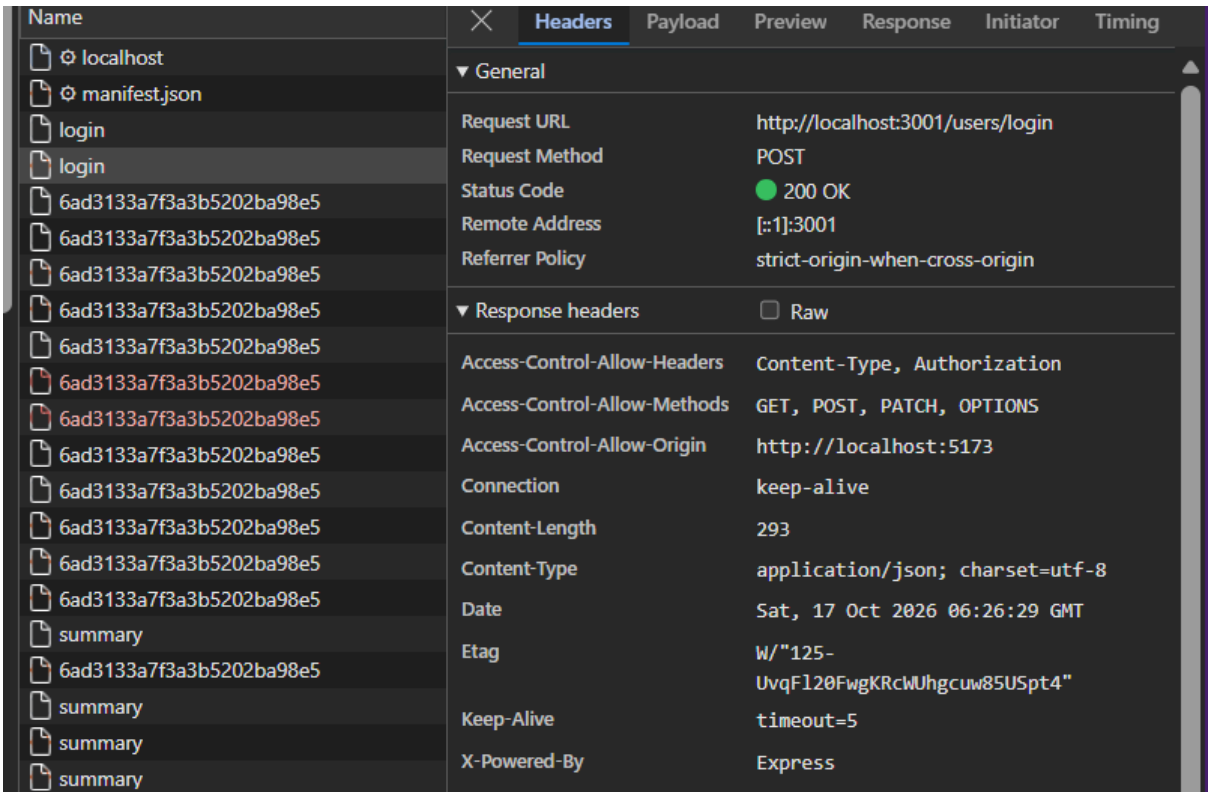
Se ejecutó una petición HTTP de tipo GET al endpoint `/users/leaderboard` del microservicio User Service mediante PowerShell, con el propósito de comprobar la recuperación de los usuarios registrados en FIXIT. La respuesta permitió identificar dos usuarios de laboratorio, `test_lab_user` y `test_lab_from_pwa_web`, cada uno con un identificador único y estadísticas iniciales en cero. Asimismo, se confirmó que la API devolvió un total de dos registros, evidenciando el funcionamiento de las operaciones de consulta y recuperación de información del backend.



Se realizó una consulta directamente desde la interfaz de MongoDB Atlas sobre la colección `users` de la base de datos `fixit_users_lab`, aplicando un filtro mediante el operador `$in` para localizar los usuarios `test_lab_user` y `test_lab_from_pwa_web`. Como

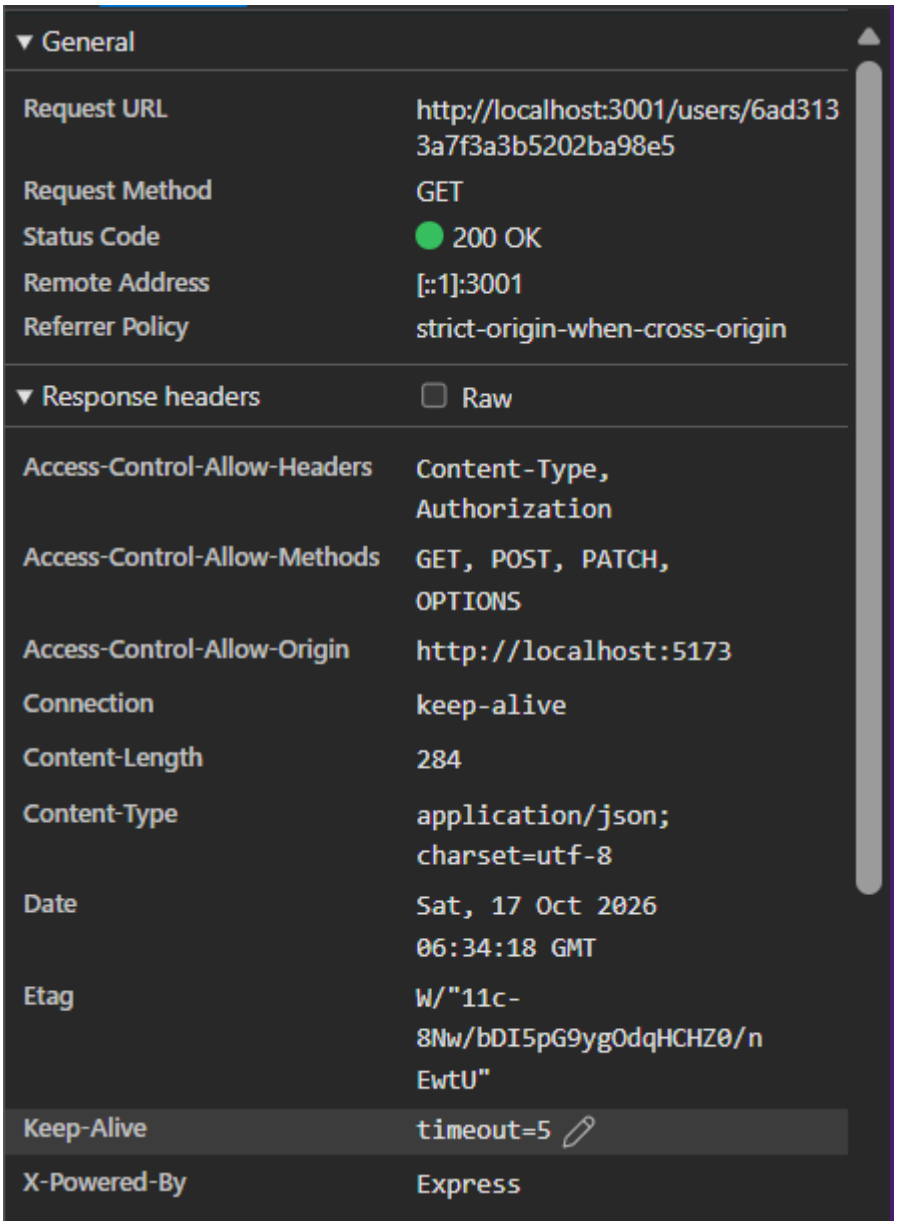
resultado, se identificaron dos documentos almacenados, cuyos identificadores únicos (`_id`) coincidieron con los obtenidos previamente mediante el endpoint GET `/users/leaderboard` del microservicio User Service. Esta comprobación confirma la persistencia de ambos registros en MongoDB Atlas y la correspondencia entre la información almacenada y los datos recuperados a través de la API REST de FIXIT.

Figura 2.2. Verificación de autenticación y recuperación de usuario mediante la PWA



Se realizó una prueba de autenticación desde la aplicación web progresiva FIXIT mediante una petición HTTP de tipo POST al endpoint `/users/login` del microservicio User Service. A través de las herramientas de desarrollo del navegador se verificó que la solicitud obtuvo el código de estado `200 OK`, confirmando su procesamiento satisfactorio. Asimismo, se comprobó que el servidor permitió las solicitudes provenientes de la PWA alojada en `http://localhost:5173` mediante la configuración de CORS y devolvió una respuesta en formato JSON. Esta evidencia demuestra la comunicación funcional entre la interfaz de la PWA y el servicio de autenticación del backend.

Figura 2.2.1. Verificación de recuperación del perfil de usuario mediante la API REST



Se verificó la recuperación de información del usuario autenticado mediante una petición HTTP de tipo GET al endpoint `/users/:id` del microservicio User Service. A través de las herramientas de desarrollo del navegador se comprobó que la solicitud realizada desde la PWA obtuvo una respuesta satisfactoria con código de estado `200 OK` y formato JSON. Asimismo, se confirmó que el identificador utilizado en la consulta correspondía al usuario `test_lab_from_pwa_web`, previamente registrado y localizado en MongoDB Atlas. Esta evidencia demuestra la comunicación entre la PWA y el backend para consultar información persistida de los usuarios.

Figura 2.3. Consulta del estado inicial de sesiones mediante la API REST

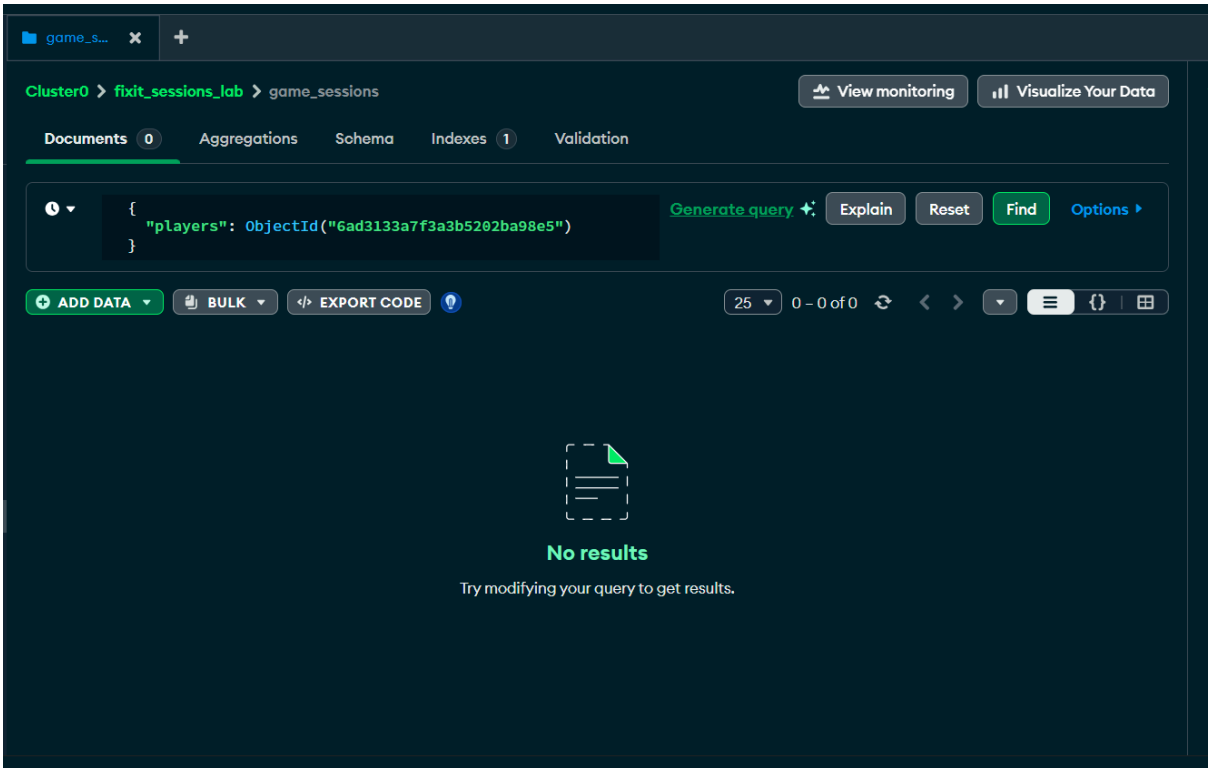
```
PS C:\Users\oscar\Desktop\asbeds\-FIXIT-Proyecto-Integrador> Invoke-RestMethod `
>>     -Uri "http://localhost:3003/sessions/user/$userId/summary" `
>>     -Method Get |
>>     ConvertTo-Json -Depth 10
{
    "repairTypes":  {

                    },
    "weekly":  [
                   0,
                   0,
                   0,
                   0,
                   0,
                   0,
                   0
               ],
    "recent":  [

               ]
}
```

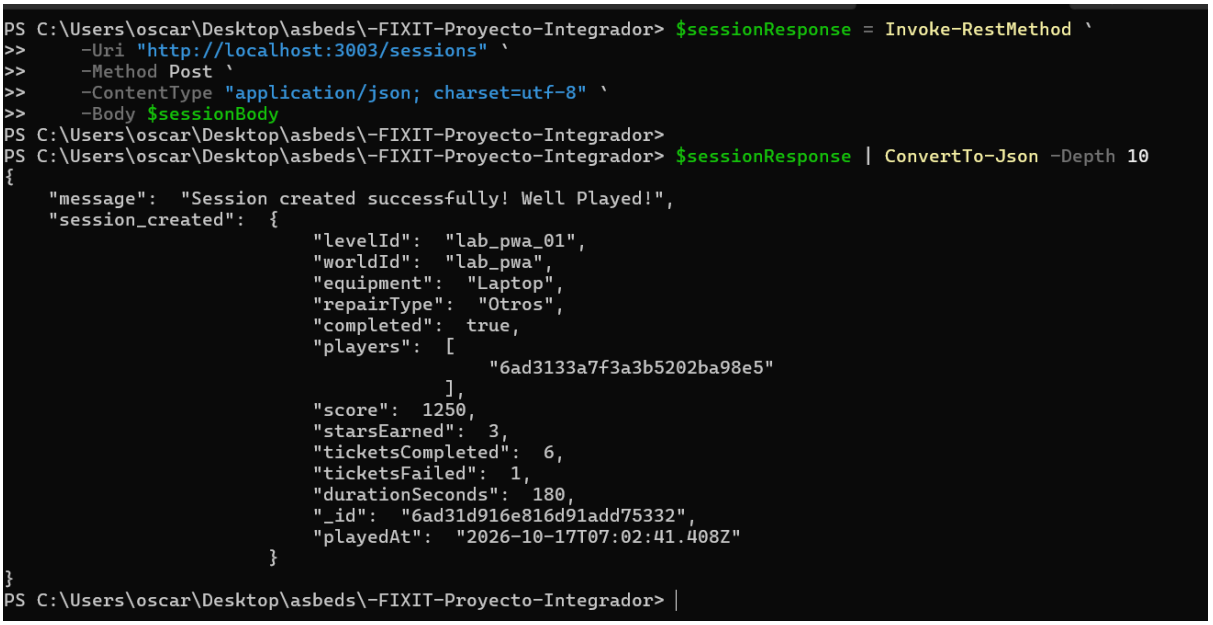
Se realizó una consulta HTTP de tipo GET al endpoint `/sessions/user/:userId/summary` del microservicio Session Service mediante PowerShell, utilizando el identificador del usuario de laboratorio `test_lab_from_pwa_web`. La respuesta devolvió una distribución de reparaciones vacía, siete valores de actividad semanal en cero y una lista de sesiones recientes sin registros. Estos resultados establecen el estado inicial del usuario antes de registrar sesiones de juego y permiten disponer de una referencia para comprobar posteriormente los cambios producidos en la información persistida.

Figura 2.3.1. Comprobación del estado inicial en MongoDB Atlas



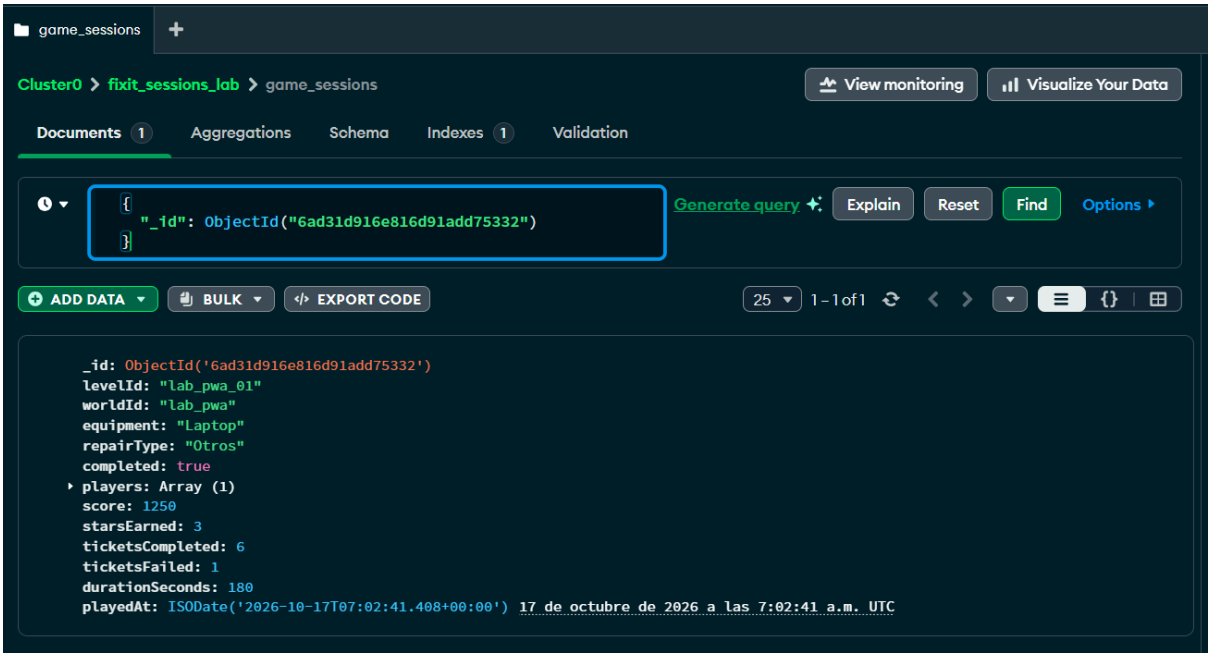
Se realizó una consulta en MongoDB Atlas sobre la colección `game_sessions` de la base de datos `fixit_sessions_lab`, utilizando el identificador del usuario `test_lab_from_pwa_web` como criterio de búsqueda en el campo `players`. La consulta no devolvió documentos, confirmando la ausencia de sesiones asociadas a dicho usuario. Este resultado fue consistente con el resumen vacío obtenido previamente mediante Session Service, estableciendo una referencia para verificar la inserción y recuperación de sesiones durante las siguientes pruebas.

Figura 2.3.2. Registro de una sesión de juego mediante la API REST de FIXIT



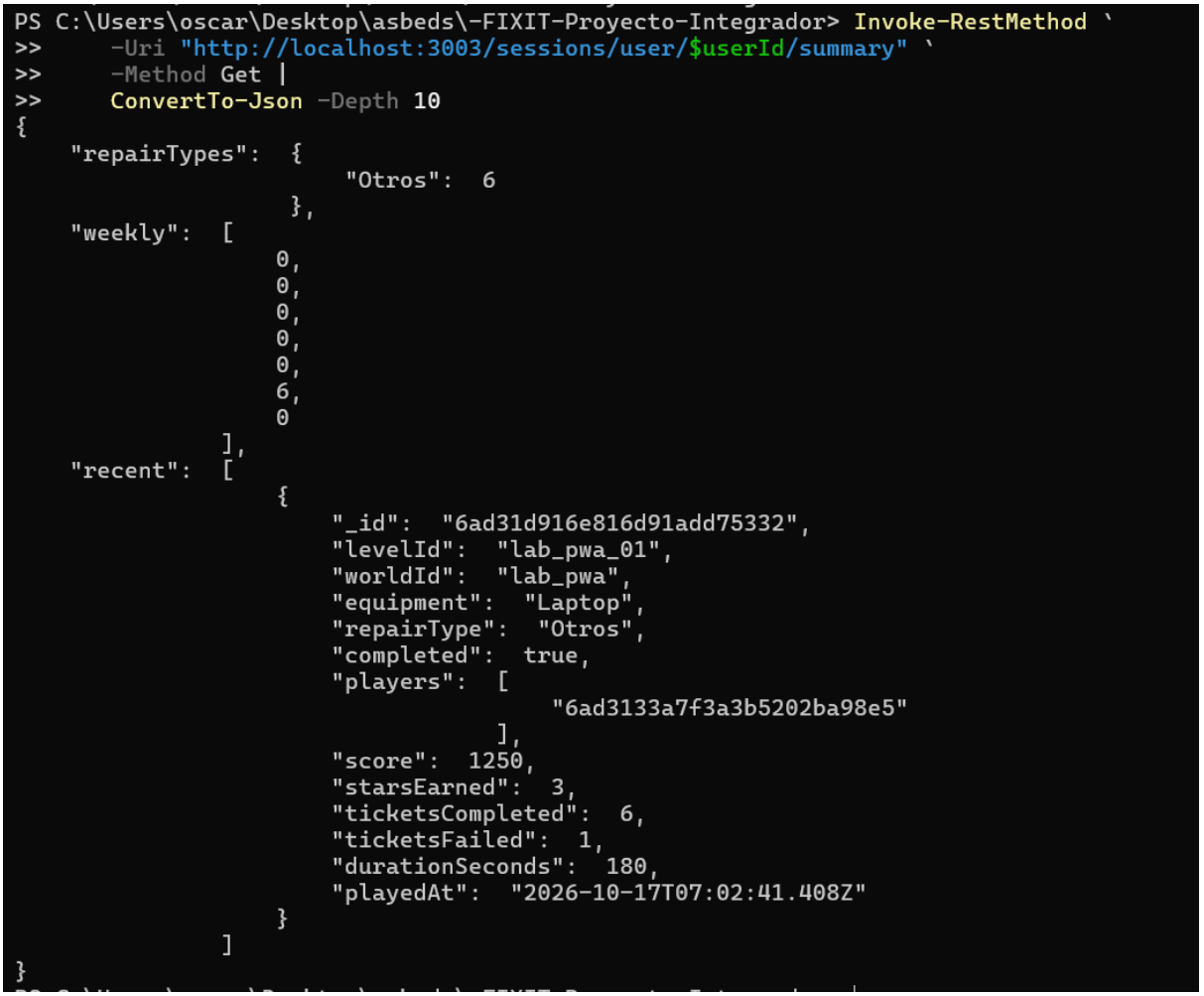
Se realizó una petición HTTP de tipo POST al endpoint `/sessions` del microservicio Session Service mediante PowerShell, con el propósito de registrar una sesión de juego asociada al usuario `test_lab_from_pwa_web`. La solicitud incluyó el identificador del nivel, el mundo, la puntuación obtenida, las estrellas, los tickets completados y fallidos, así como la duración de la partida. La API devolvió un mensaje de confirmación y el documento generado con un identificador único (`_id`) y una fecha de registro (`playedAt`). Esta evidencia confirma la creación de la sesión mediante la API REST y permite identificar el documento para su posterior comprobación en MongoDB Atlas.

Figura 2.3.3. Comprobación de la persistencia de una sesión de juego en MongoDB Atlas



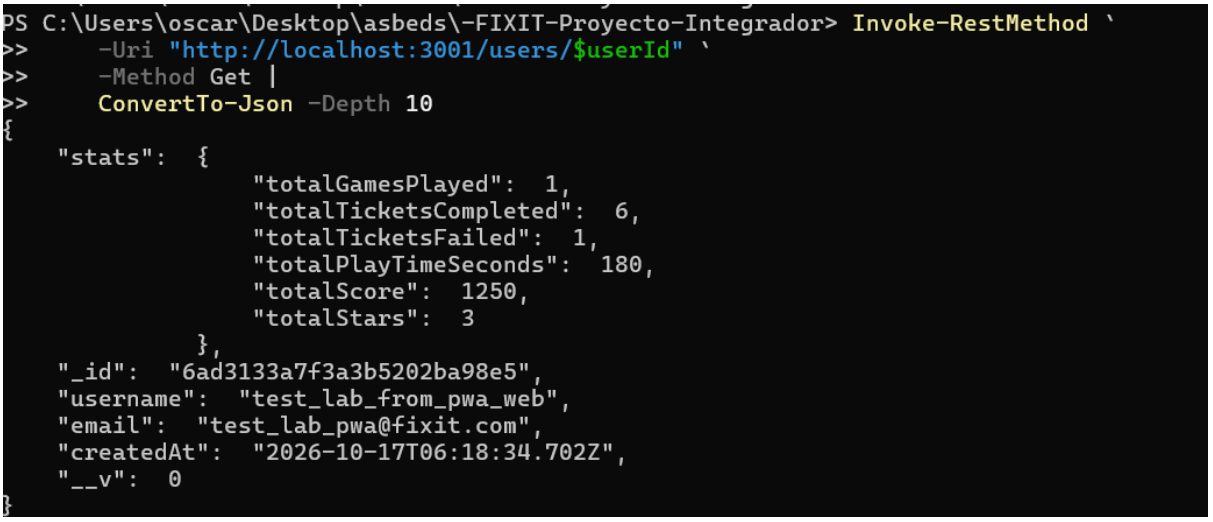
Se realizó una consulta directamente en MongoDB Atlas sobre la colección `game_sessions` de la base de datos `fixit_sessions_lab`, utilizando el identificador único (`_id`) generado previamente por el endpoint POST `/sessions`. La consulta permitió recuperar un documento correspondiente a la sesión registrada, con una puntuación de 1250 puntos, 3 estrellas, 6 tickets completados, 1 ticket fallido y una duración de 180 segundos. Asimismo, se comprobó que el identificador y los valores almacenados coincidían con la respuesta obtenida desde Session Service, confirmando la persistencia de la información en MongoDB Atlas.

Figura 2.3.4. Verificación del resumen actualizado de sesiones mediante la API REST



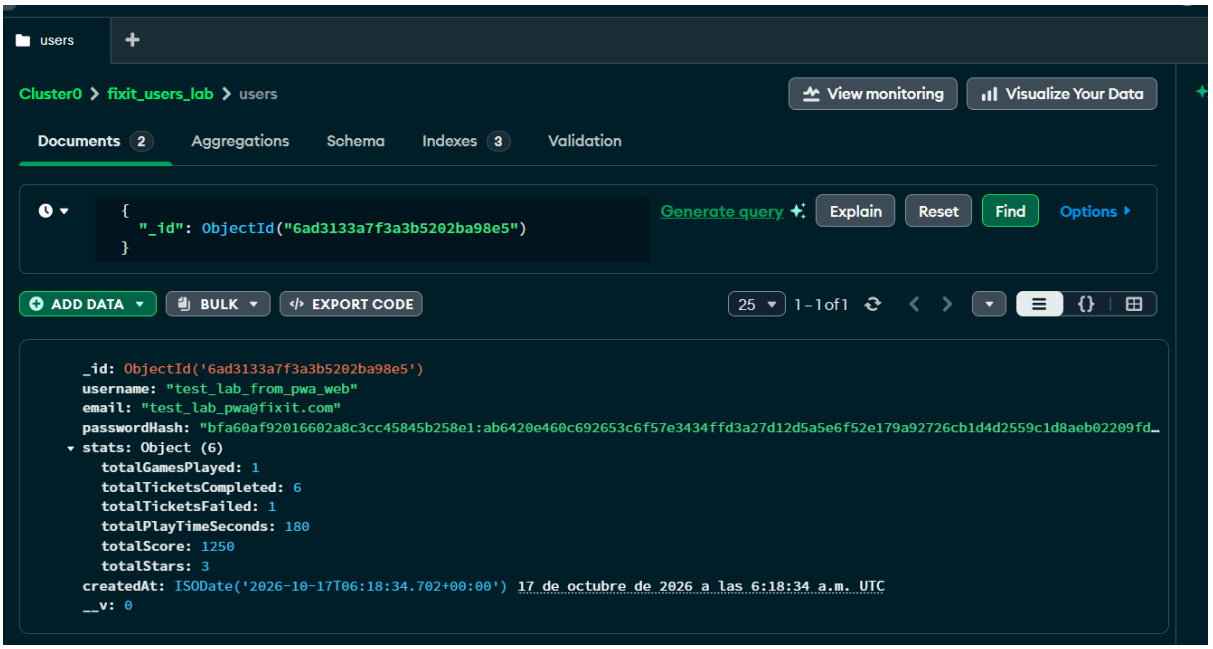
Se ejecutó una petición HTTP de tipo GET al endpoint `/sessions/user/:userId/summary` del microservicio Session Service mediante PowerShell, con el propósito de verificar la recuperación de información después de registrar una sesión de juego. La respuesta mostró seis reparaciones correspondientes a la categoría «Otros», actividad semanal con seis tickets completados y una sesión reciente asociada al usuario de laboratorio. Asimismo, se recuperaron los datos originales de la partida, incluyendo una puntuación de 1250 puntos, tres estrellas y una duración de 180 segundos. Estos resultados, comparados con el estado inicial sin actividad, demuestran que la API REST recupera correctamente la información persistida en MongoDB Atlas.

Figura 2.4. Verificación de estadísticas actualizadas mediante User Service



Se ejecutó una petición HTTP de tipo GET al endpoint `/users/:id` del microservicio User Service mediante PowerShell, con el propósito de verificar la actualización de las estadísticas del usuario después del registro de una sesión de juego. La respuesta permitió identificar una partida jugada, seis tickets completados, un ticket fallido, 180 segundos de actividad, 1250 puntos y tres estrellas acumuladas. Los seis valores coincidieron con los incrementos esperados a partir de la sesión registrada previamente mediante Session Service, evidenciando la comunicación entre ambos microservicios y la recuperación de las estadísticas actualizadas mediante la API REST.

Figura 2.4.1. Comprobación de estadísticas persistidas en MongoDB Atlas



Se realizó una consulta directamente en MongoDB Atlas sobre la colección `users` de la base de datos `fixit_users_lab`, utilizando el identificador único del usuario

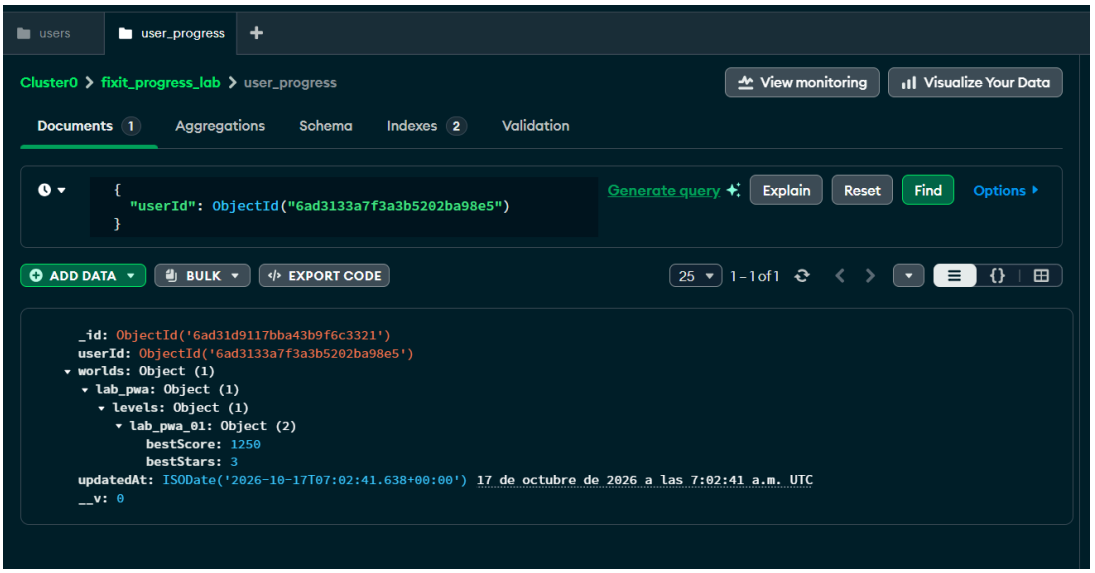
`test_lab_from_pwa_web` como criterio de búsqueda. La consulta permitió recuperar el documento correspondiente y examinar el objeto `stats`, donde se identificaron una partida jugada, seis tickets completados, un ticket fallido, 180 segundos de actividad, 1250 puntos y tres estrellas acumuladas. Estos valores coincidieron con los obtenidos previamente mediante el endpoint `GET /users/:id`, confirmando la persistencia de las estadísticas actualizadas y la correspondencia entre los datos almacenados y la información proporcionada por User Service.

Figura 2.4.2. Consulta del progreso del usuario mediante Progress Service



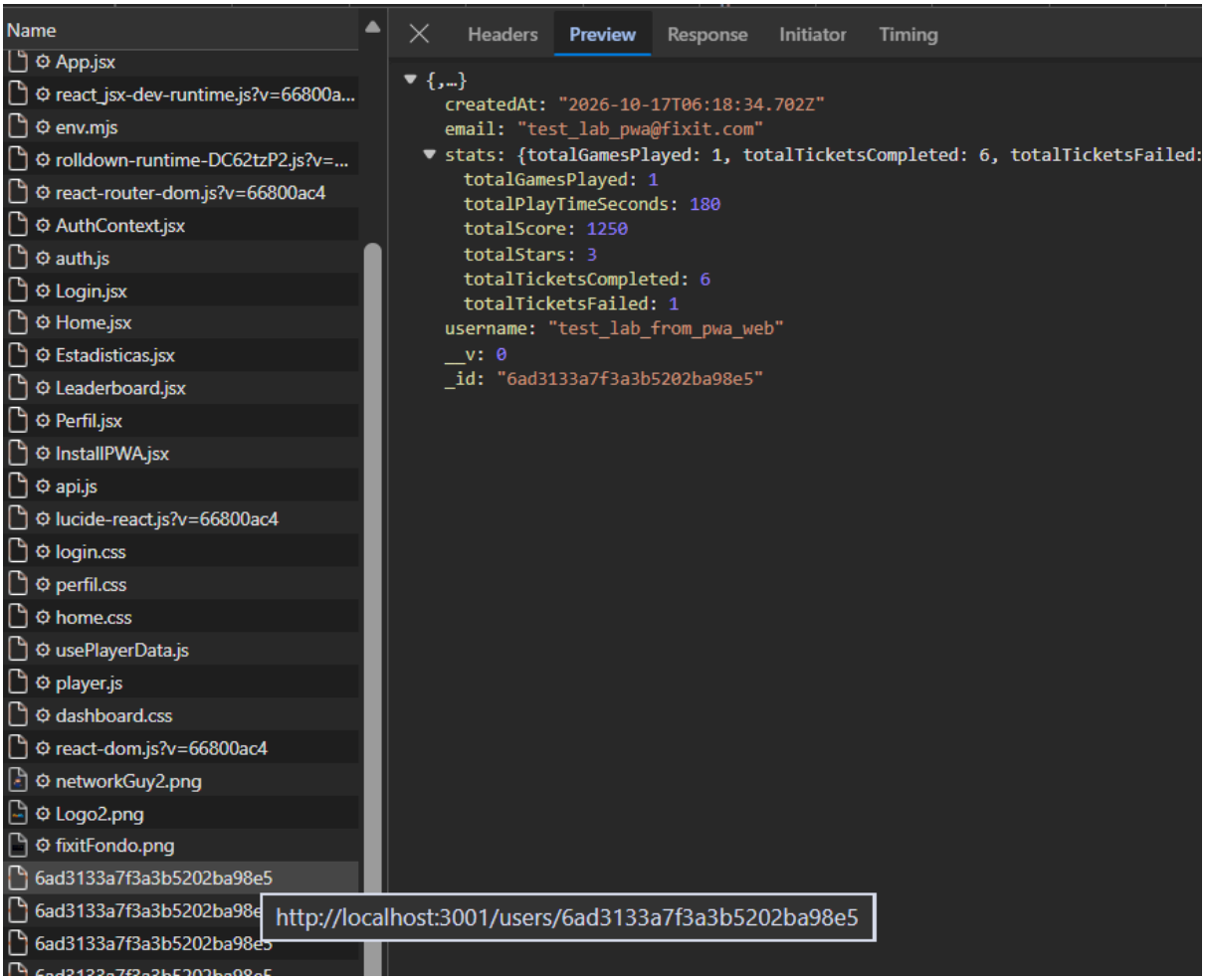
Se realizó una petición HTTP de tipo GET al endpoint `/progress/:userId` del microservicio Progress Service mediante PowerShell, con el propósito de verificar la recuperación del progreso generado después del registro de una sesión de juego. La respuesta devolvió un documento asociado al usuario `test_lab_from_pwa_web`, que contenía el mundo `lab_pwa` y el nivel `lab_pwa_01`, con una puntuación máxima (`bestScore`) de 1250 puntos y una cantidad máxima de estrellas (`bestStars`) de tres. Estos valores coincidieron con los registrados previamente mediante Session Service, evidenciando la actualización del progreso del usuario y su recuperación satisfactoria mediante la API REST.

Figura 2.4.3. Comprobación del progreso persistido en MongoDB Atlas



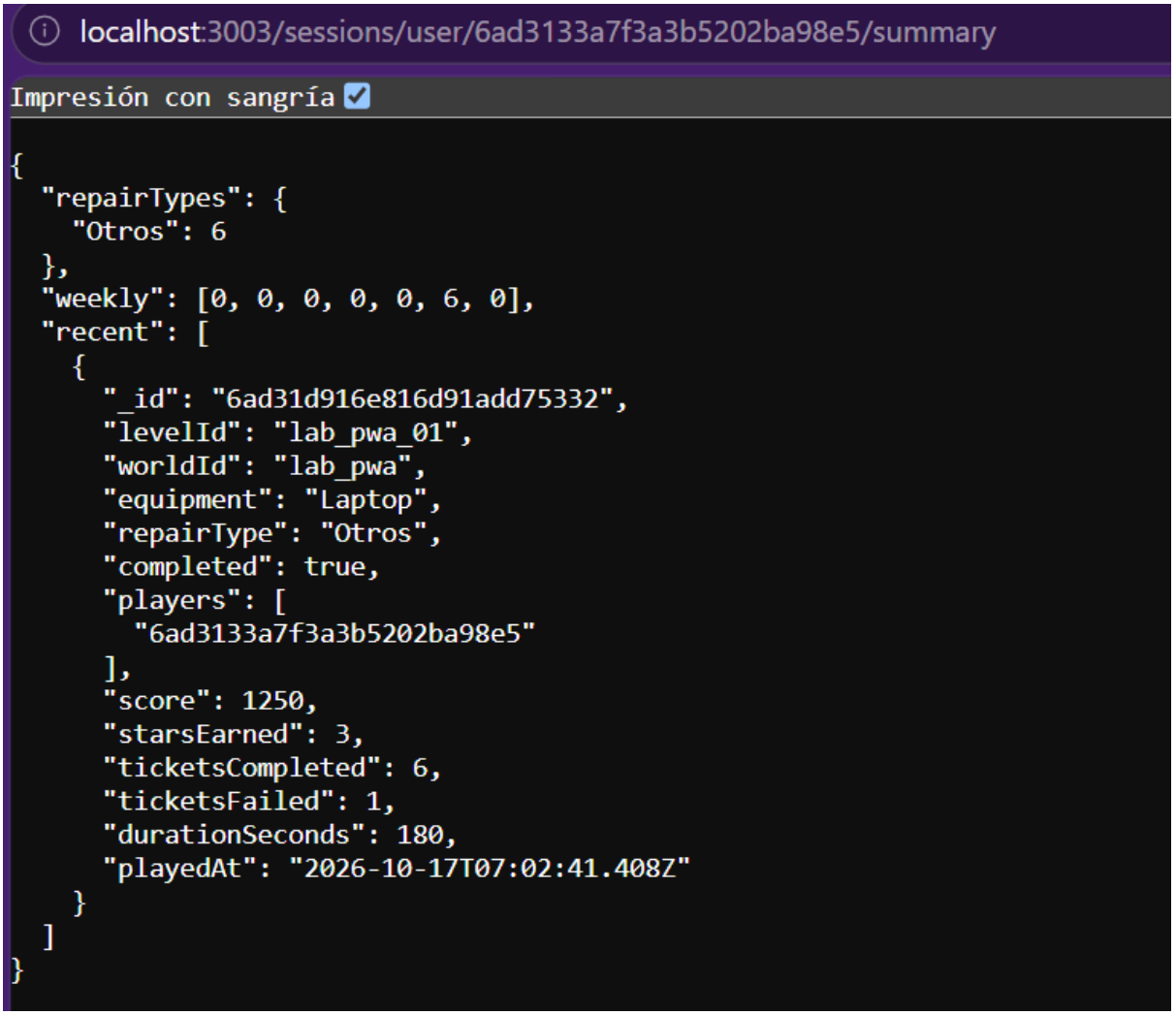
Se realizó una consulta directamente en MongoDB Atlas sobre la colección `user_progress` de la base de datos `fixit_progress_lab`, utilizando el identificador del usuario `test_lab_from_pwa_web` como criterio de búsqueda. La consulta permitió localizar un documento que contenía el mundo `lab_pwa` y el nivel `lab_pwa_01`, con una puntuación máxima (`bestScore`) de 1250 puntos y tres estrellas (`bestStars`). Asimismo, se verificó que el identificador del usuario y los valores almacenados coincidían con los obtenidos previamente mediante el endpoint `GET /progress/:userId` de Progress Service. Esta evidencia confirma la persistencia del progreso y la correspondencia entre los datos almacenados y la respuesta de la API REST.

Figura 2.5. Verificación de la recuperación de estadísticas persistidas desde la PWA FIXIT



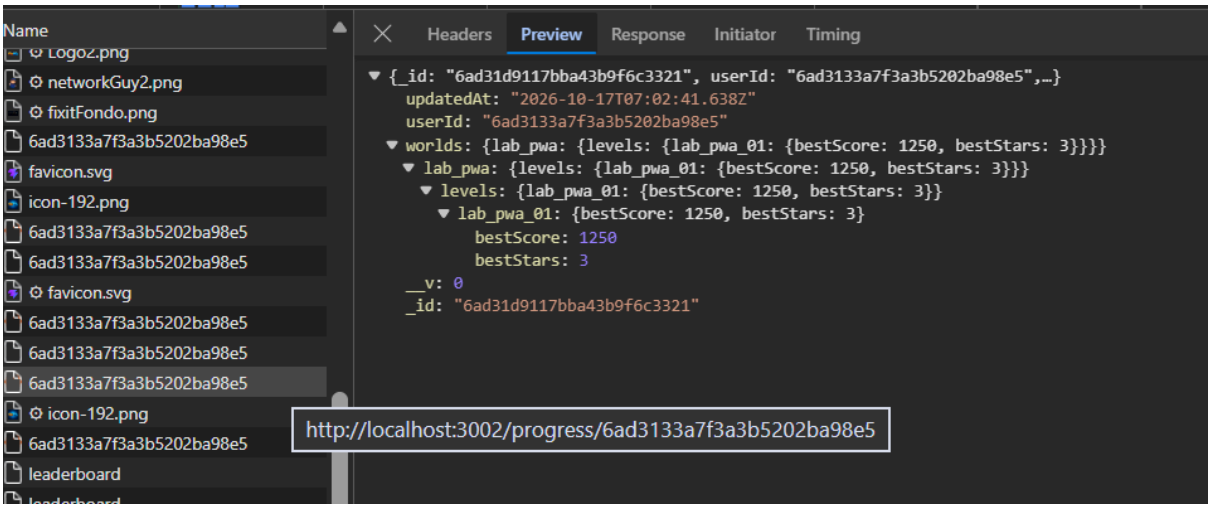
Se verificó la recuperación de estadísticas del usuario desde la aplicación web progresiva FIXIT mediante las herramientas de desarrollo del navegador. En la pestaña Network se inspeccionó la respuesta de una petición HTTP al endpoint `/users/:id` del microservicio User Service, identificando una partida jugada, seis tickets completados, un ticket fallido, 180 segundos de actividad, 1250 puntos y tres estrellas acumuladas. Estos valores coincidieron con los registros comprobados previamente en MongoDB Atlas y con la información presentada en la sección de perfil de la PWA. La evidencia demuestra la integración funcional entre la interfaz web, la API REST y los datos persistidos del usuario.

Figura 2.5.2. Verificación de la recuperación del resumen de sesiones desde la PWA FIXIT



Se verificó la consulta del resumen de sesiones de juego realizada desde la aplicación web progresiva FIXIT hacia el microservicio Session Service, mediante el endpoint GET `/sessions/user/:userId/summary`. La respuesta obtenida presentó seis reparaciones de la categoría «Otros», actividad semanal correspondiente a seis tickets completados y una sesión reciente asociada al usuario de laboratorio. Asimismo, se recuperaron los datos de la partida registrada, incluyendo 1250 puntos, tres estrellas y una duración de 180 segundos. La información coincidió con los documentos previamente verificados en MongoDB Atlas, evidenciando la correcta recuperación de datos persistidos mediante la API REST.

Figura 2.5.3. Verificación de la recuperación del progreso del usuario desde la PWA FIXIT



Se verificó la recuperación del progreso del usuario desde la aplicación web progresiva FIXIT mediante las herramientas de desarrollo del navegador. En la pestaña Network se identificó una solicitud al endpoint `/progress/:userId` del microservicio Progress Service, cuya respuesta contenía el documento asociado al usuario `test_lab_from_pwa_web`. Dentro de la estructura `worlds` se identificó el mundo `lab_pwa` y el nivel `lab_pwa_01`, con una puntuación máxima (`bestScore`) de 1250 puntos y tres estrellas (`bestStars`). Los valores coincidieron con los comprobados previamente mediante la API REST y MongoDB Atlas, demostrando que la PWA recupera correctamente la información persistida del progreso del usuario.